

Open League Analyzer (OLA) Player Performance Data Analysis

Technical Report

Mateusz Piławski

OLA Repo

July 9, 2026

Introduction

This document outlines the technical aspects and methodology behind creating a player performance metric for the Open League Analyzer (OLA). The rationale and technical details are explained as comprehensively as possible. I have authored this report as an entry-level data analyst taking my first steps into the field of data science. Fundamentally, this is a data exploration project aimed at extracting actionable insights from a vast dataset of League of Legends (LoL) players. Utilizing the CRISP-DM framework [1], I navigated the Riot API to extract and prepare data that yields the highest value within the constraints of my available resources.

Disclaimer: I do not guarantee that the methodologies and actions described below represent industry best practices. If you have suggestions for improvement or valuable insights regarding LoL data and this project, please feel free to reach out to me on LinkedIn.

Business and Game Understanding

To understand the technical metrics established in this report, it is necessary to outline the core operational mechanics of League of Legends (LoL), the structure of its data ecosystem, and the business model of performance-analytics platforms such as OP.GG.

League of Legends and the Competitive Ecosystem

League of Legends is a multiplayer online battle arena (MOBA) game driven by a highly structured competitive matchmaking system. Players are distributed across distinct regional servers to minimize latency, where they compete in "Ranked" ladders categorized into tiers (e.g., Iron to Challenger). The core gameplay generates high-frequency telemetry data covering mechanical actions, economic scaling (gold acquisition), objective control (towers, dragons), and combat outcomes.

The Data Landscape

Data collection is externalized primarily via the official Riot Games API. This interface exposes granular endpoints categorized into:

- **Summoner Data:** Player identity, current rank, and historical match verification.
- **Match History:** Comprehensive, post-game telemetry including item builds, skill orders, damage output metrics, and timeline vectors.
- **Live Client Data:** Real-time telemetry utilized for active match tracking.

Managing this data requires handling extreme throughput, requiring scalable ETL pipelines to clean, store, and aggregate millions of matches daily across multiple regional clusters.

Business Value and Application for Analytical Platforms

For data-aggregation platforms like OP.GG, processing this raw telemetry serves specific commercial and user-facing objectives:

- **Statistical Meta-Analysis:** Aggregating global match histories allows platforms to calculate win rates, pick/ban rates, and optimal itemization pathways. This translates raw data into actionable intelligence for players seeking optimization.
- **Player Profiling and Benchmarking:** Programmatic calculation of individual performance metrics allows users to evaluate their performance relative to regional baselines.
- **Monetization Strategy:** High-traffic analytics engines monetize their infrastructure primarily through display advertising, premium ad-free subscriptions, and data licensing to professional esports organizations requiring predictive scouting tools.

Data Acquisition

Fetching Script

To build an accurate evaluation model, relevant game data must first be acquired. During this process, several key factors were considered:

1. **Proper data representation.** Creating a unified system capable of evaluating individual player performance requires robust data representation for each cohort (in the case of LoL, these are ranks or tiers).
2. **Limited computing resources.** As this is an independent project, all computational tasks—including testing, training, and evaluating algorithms—are strictly limited by the hardware capabilities of my personal computer.

Given the structure of the LoL player base, stratified random sampling by rank is the most logical approach. The rank distribution data was sourced from DPM.LOL [2], and an initial sample size of $N = 10000$ was selected as efficient for preliminary testing.

The first attribute considered was the server region. Only servers with a ranked population exceeding one million players were included, as they provide the most statistically significant data for potential OLA users. We then calculated the relative weight of each server and the corresponding number of players to sample:

Table 1: Server Performance Metrics and Subsample Sizes

Server	Ranked Players Count	% of Total Players	Subsample Size (N=10000)
Europe West (EUW)	3 104 938	32.66%	3300
Korea (KR)	2 623 034	27.59%	2800
North America (NA)	1 372 532	14.44%	1400
Europe Nordic & East (EUNE)	1 352 534	14.23%	1400
Brazil (BR)	1 054 597	11.08%	1100

Next, using the rank distribution and the percentage of players within each tier per server, we calculated the exact number of players to fetch from each subgroup. An example breakdown for the EUW server is provided below:

Table 2: Example Sampling Distribution and Verification Matrix Across Competitive Tiers (EUW)

Total Players	Subsample Size	Tier	Amount	Amount (%)	Fetch Amount	Verifying Sum
3 104 938	3300	Iron	107 000	3.44%	113	3300
		Bronze	469 000	15.10%	498	
		Silver	664 000	21.38%	705	
		Gold	756 000	24.34%	803	
		Platinum	571 000	18.39%	606	
		Emerald	365 000	11.75%	387	
		Diamond	144 000	4.63%	153	
		Master	28 000	0.90%	29	
		Grandmaster	768	0.025%	4	
		Challenger	308	0.009%	2	

For deeper insights into the statistics across other servers, please refer to the "LoL Rank Distribution via DPM.LOL.xlsx" file in the OLA repository.

With the sample sizes established, we can proceed to sample players from each defined group. This is achieved using a semi-random sampling method, generating two integers: the first represents a page number, and the second represents a specific player's index on that page. The sampling is executed against the OP.GG leaderboards. Fortunately, the fetching script minimizes the number of initial players we need to sample manually. The script automatically gathers the co-players from the initial summoner's match history, filtering them to ensure they match the original summoner's rank and server.

References

- [1] Rüdiger Wirth and Jochen Hipp. "CRISP-DM: Towards a Standard Process Model for Data Mining". en. In: ().
- [2] *DPM.LOL - League of Legends stats best build and tierlist*. en. URL: <https://dpm.lol/studio/ranks> (visited on 07/09/2026).